

RAG - 4 (Split)

1. Split 단계가 필요한 이유

Load 단계에서 문서를 불러왔다고 해서 바로 좋은 검색이 되는 것은 아닙니다. 문서를 너무 큰 덩어리로 그대로 저장하면, 질문과 직접 관련된 부분만 정확하게 찾기 어렵습니다. 반대로 문서를 너무 작게 나누면 문맥이 끊겨서 의미가 약해질 수 있습니다.

예를 들어 30페이지짜리 PDF 전체를 하나의 문서로 저장하면, 검색기는 그 안에서 어떤 부분이 중요한지 정확하게 찾기 어렵습니다. 반대로 한두 문장 단위로 지나치게 잘게 나누면, 각 조각이 가진 정보량이 너무 적어서 질문에 충분한 맥락을 제공하지 못할 수 있습니다.

따라서 Split 단계는 단순히 문서를 잘라내는 작업이 아니라, **검색에 적합한 크기로 문서를 재구성하는 과정**이라고 이해하는 것이 중요합니다.

쉽게 말하면 Split은 다음 질문에 답하는 과정입니다.

문서를 어느 정도 길이로 나누어야
검색이 잘 되고, 문맥도 유지될까?

2. Split의 핵심 개념

2-1. Chunk

Chunk는 문서를 나눈 하나의 조각입니다.

RAG에서는 이 chunk 단위로 임베딩을 만들고, 검색도 이 단위로 수행하게 됩니다.

즉, 문서 전체를 한 번에 검색하는 것이 아니라,

문서를 여러 개의 작은 의미 단위로 나눈 후 그 조각들 중에서 질문과 관련된 부분을 찾는 방식입니다.

2-2. Chunk Size

Chunk Size는 각 chunk의 최대 길이를 의미합니다.

예를 들어 `chunk_size=1000` 이라면,

문서를 1000자 내외의 크기로 나누겠다는 뜻입니다.

Chunk Size가 너무 크면 한 조각 안에 너무 많은 내용이 들어가고,

너무 작으면 문맥이 지나치게 잘려 나갈 수 있습니다.

2-3. Chunk Overlap

Chunk Overlap은 앞 chunk와 뒤 chunk가 서로 일부 내용을 겹쳐 가지도록 만드는 값입니다.

예를 들어 `chunk_overlap=200` 이라면,

앞 chunk의 마지막 200자가 다음 chunk의 앞부분에도 다시 포함됩니다.

이 overlap은 문장이나 문단이 경계에서 끊기면서 의미가 손실되는 문제를 줄여줍니다.

3. 왜 Overlap이 필요한가?

문서는 항상 딱 잘라 떨어지는 구조로 쓰이지 않습니다.

한 문장이 다음 문장과 연결되고, 한 문단의 핵심 내용이 다음 부분으로 이어지는 경우가 많습니다.

만약 overlap 없이 문서를 자르면, 중요한 설명이 두 chunk 사이에 갈라져 저장될 수 있습니다.

그러면 검색할 때 관련 정보가 충분히 함께 검색되지 않을 가능성이 생깁니다.

예를 들어 다음과 같은 문장이 있다고 가정해봅시다.

┆ RAG는 외부 문서를 검색한 뒤, 그 결과를 바탕으로 답변을 생성한다.

이 문장이 chunk 경계에서 나뉘면 앞 chunk에는 “외부 문서를 검색한 뒤”만 남고,

뒤 chunk에는 “그 결과를 바탕으로 답변을 생성한다”만 남을 수 있습니다.

이렇게 되면 각각의 chunk가 온전한 의미를 가지기 어려워집니다.

따라서 overlap을 주면 경계 부근의 중요한 문맥을 더 안정적으로 유지할 수 있습니다.

4. LangChain에서 가장 먼저 배우는 Splitter

LangChain에서 일반 텍스트를 나눌 때 가장 먼저 배우는 분할기는 보통

`RecursiveCharacterTextSplitter` 입니다.

이 splitter는 단순히 글자 수만 세서 자르는 것이 아니라,

가능하면 문단과 줄바꿈, 단어 경계를 최대한 보존하면서 나누려고 합니다.

즉, 다음과 같은 순서로 나누기를 시도합니다.

1. 문단 단위

2. 줄바꿈 단위
3. 공백 단위
4. 마지막에는 문자 단위

이 방식의 장점은 문서를 무작정 잘라내는 것이 아니라,
가능한 한 자연스러운 의미 단위를 유지하면서 나눌 수 있다는 점입니다.

문서를 기계적으로 자르는 것이 아니라,
가능한 한 문단과 문장의 흐름을 살려서 나누는 기본 분할기

5. 설치

텍스트 분할기를 사용하려면 다음 패키지를 설치합니다.

```
pip install -U langchain-text-splitters
```

6. 가장 기본적인 Split 예제

다음은 문자열 자체를 직접 나누는 가장 단순한 예제입니다.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text = """
RAG는 외부 문서를 검색한 뒤,
그 결과를 바탕으로 답변을 생성하는 방식입니다.
문서를 너무 크게 두면 검색 정확도가 떨어질 수 있습니다.
그래서 문서를 적절한 크기의 chunk로 나누는 과정이 중요합니다.
"""

splitter = RecursiveCharacterTextSplitter(
    chunk_size=50,
    chunk_overlap=10
)

chunks = splitter.split_text(text)
```

```
for i, chunk in enumerate(chunks):
    print(f"[Chunk {i}]")
    print(chunk)
    print("-" * 30)
```

이 코드는 하나의 긴 문자열을 여러 개의 chunk로 나누는 예제입니다.

- `chunk_size=50`
각 chunk의 최대 길이를 50자로 제한합니다.
- `chunk_overlap=10`
앞 chunk의 마지막 10자가 뒤 chunk에도 일부 포함되도록 합니다.

7. Load한 문서를 그대로 Split하는 예제

실제 RAG에서는 보통 문자열을 직접 자르기보다,

Load 단계에서 불러온 `Document` 객체를 그대로 나누게 됩니다.

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = TextLoader("sample.txt", encoding="utf-8")
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

split_docs = splitter.split_documents(docs)

print("원본 문서 수:", len(docs))
print("분할 후 문서 수:", len(split_docs))
print(split_docs[0].page_content[:300])
print(split_docs[0].metadata)
```

이 코드는 앞 시간에 배운 Load 단계와 이번 시간의 Split 단계를 자연스럽게 연결해 줍니다.

흐름은 다음과 같습니다.

1. 텍스트 파일을 불러온다.
 2. 불러온 문서를 splitter로 나눈다.
 3. 나뉜 chunk를 확인한다.
-

8. 기본값은 어떻게 정하면 좋을까?

처음 Split을 연습할 때 가장 많이 하는 질문은

“chunk size를 얼마로 해야 하나요?”입니다.

정답이 하나로 고정되어 있는 것은 아니지만,
입문 단계에서는 다음과 같이 시작하면 무난합니다.

- 짧은 실습용 텍스트: 200 ~ 500
- 일반 문서나 PDF: 800 ~ 1200
- overlap: 100 ~ 200

처음에는 너무 복잡하게 생각하기보다,
적당한 크기에서 시작해서 직접 결과를 비교해보는 것이 중요합니다.
즉, Split은 공식 하나를 외우는 것이 아니라
문서를 보고 조절하는 감각을 익히는 과정이라고 볼 수 있습니다.

9. Chunk Size가 너무 크면 생기는 문제

Chunk가 너무 크면 한 조각 안에 너무 많은 정보가 들어갑니다.

이 경우 질문과 직접 관련 없는 문장도 함께 들어오게 되어 검색 정확도가 떨어질 수 있습니다.

예를 들어 하나의 chunk 안에 여러 주제가 한꺼번에 들어 있으면,
검색 결과가 나와도 정확히 어떤 부분이 답변에 필요한 내용인지 모호해질 수 있습니다.
또한 너무 큰 chunk는 이후 모델에 전달할 때 비용과 입력 길이 측면에서도 불리할 수 있습니다.

10. Chunk Size가 너무 작으면 생기는 문제

반대로 chunk가 너무 작으면 문맥이 부족해질 수 있습니다.

예를 들어 한두 문장만 남는 수준으로 너무 짧게 자르면,

검색은 되었더라도 질문에 충분한 설명을 제공하지 못할 수 있습니다.

특히 정의, 이유, 예시처럼 서로 이어진 설명은

너무 잘게 나누면 의미가 약해집니다.

따라서 Split의 핵심은 다음과 같습니다.

- 너무 크게도 자르지 말고
- 너무 작게도 자르지 않으며
- 의미 단위를 가능한 한 유지하는 것

11. Markdown 문서는 어떻게 나누는가?

Markdown 문서는 일반 텍스트와 달리 제목 구조가 분명한 경우가 많습니다.

예를 들어 다음과 같은 구조가 있을 수 있습니다.

- # 1장
- ## 1-1
- ### 예시

이런 문서는 단순히 글자 수만 기준으로 자르기보다,

먼저 제목 기준으로 문서를 구분한 뒤, 필요하면 그 안에서 다시 세부 분할하는 것이 더 자연스러울 수 있습니다.

즉, Markdown 문서는 보통 다음 순서로 생각하면 좋습니다.

1. 제목 구조를 먼저 기준으로 나눈다.
2. 너무 긴 부분만 다시 잘게 나눈다.

이 방법은 수업자료, 기술문서, README 같은 구조화된 문서에서 특히 유용합니다.

12. 코드 문서는 어떻게 나누는가?

코드 문서는 일반 설명문과 다릅니다.

함수, 클래스, 조건문, 반복문 같은 구조가 중요하기 때문입니다.

만약 코드를 일반 텍스트처럼 아무 데서나 잘라버리면,

하나의 함수가 두 개의 chunk로 쪼개지거나 클래스 정의가 중간에서 끊길 수 있습니다. 그래서 코드 문서는 가능한 경우 언어 구조를 반영한 splitter를 사용하는 것이 좋습니다. LangChain에서는 언어별 splitter 개념도 지원하므로, Python 코드라면 Python 구조에 맞춰 좀 더 자연스럽게 나누는 방식으로 접근할 수 있습니다.

13. 코드 분할 예제

```
from langchain_text_splitters import Language, RecursiveCharacterTextSplitter

code = """
def greet(name):
    return f"Hello, {name}"

def add(a, b):
    return a + b

class User:
    def __init__(self, name):
        self.name = name
"""

splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=80,
    chunk_overlap=10
)

chunks = splitter.split_text(code)

for i, chunk in enumerate(chunks):
    print(f"[Chunk {i}]")
    print(chunk)
    print("-" * 30)
```

이 예제는 Python 코드 구조를 고려하여 chunk를 나누는 방식입니다.

14. Split 단계에서 가장 중요한 관점

Split 단계에서 가장 중요한 것은

단순히 많이 자르는 것이 아니라 **의미를 유지한 채 검색 가능한 단위로 만드는 것**입니다.

즉, 좋은 Split은 다음 조건을 만족해야 합니다.

- 질문과 관련된 정보를 잘 찾을 수 있어야 한다.
- 하나의 chunk가 너무 많은 주제를 담지 않아야 한다.
- chunk가 너무 짧아 의미가 사라지지 않아야 한다.
- 문맥이 경계에서 끊기지 않도록 overlap을 적절히 사용해야 한다.

이 관점을 이해하면, 이후 어떤 문서를 다루더라도

적절한 splitter와 설정값을 선택하는 기준을 세울 수 있습니다.

15. 학습 정리

이번 시간에는 문서를 검색하기 좋은 단위로 나누는 Split 단계를 학습했습니다.

핵심 내용을 정리하면 다음과 같습니다.

- Split은 문서를 검색 가능한 chunk로 나누는 과정이다.
- chunk size는 각 조각의 길이를 결정한다.
- chunk overlap은 경계에서 문맥이 끊기는 문제를 줄여준다.
- 일반 텍스트에는 `RecursiveCharacterTextSplitter`가 기본적으로 많이 사용된다.
- Markdown과 코드 문서는 구조를 고려한 분할이 더 적합할 수 있다.

즉, 좋은 RAG를 만들기 위해서는

문서를 불러오는 것뿐 아니라 **어떻게 나눌 것인지**도 매우 중요합니다.

16. 실습 활동

실습 1

짧은 문자열을 `RecursiveCharacterTextSplitter`로 나누고,

chunk 결과를 직접 출력해보세요.

실습 2

같은 텍스트에 대해 `chunk_overlap=0` 과 `chunk_overlap=20` 을 각각 적용해
결과 차이를 비교해보세요.

실습 3

Load 단계에서 불러온 텍스트 파일 또는 PDF를 나눈 뒤,
원본 문서 수와 분할 후 문서 수를 비교해보세요.

실습 4

Markdown 문서 또는 코드 문서를 준비한 뒤,
일반 텍스트 분할과 구조 기반 분할 중 어떤 방식이 더 적절한지 토의해보세요.

17. 확인 문제

1. Split 단계가 필요한 이유는 무엇인가?
 2. Chunk overlap은 왜 필요한가?
 3. Chunk size가 너무 크면 어떤 문제가 생기는가?
 4. Chunk size가 너무 작으면 어떤 문제가 생기는가?
 5. Markdown 문서와 코드 문서는 왜 일반 텍스트와 다르게 분할할 필요가 있는가?
-